

Gestión de datos en Streaming

Adrián Robles Arques

Introducción

En este segundo sprint vamos a aprender a manejar datos empleando la herramienta Kafka como herramienta de ingesta y gestión de ficheros en streaming. En este caso vamos a cargar un conjunto de datos grande de datos, para que sea dividido en base a ventanas de eventos y gestionado mediante HDFS, pudiendo realizar consultas con otras tablas de datos anteriormente implementadas.

Consideraciones generales

Para el presente trabajo es imprescindible haber creado y cargado adecuadamente los datos en la tabla “aeropuertos” empleada durante el Sprint anterior. Para ello me he asegurado previamente de que la tabla está presente en el sistema de ficheros y los datos siguen siendo accesibles, lo que ha requerido volver a cargar los datos y la tabla para asegurar que funcionase adecuadamente.

Por otra parte, la nueva tabla que crearemos parte de una base de datos de aerolíneas que presenta la siguiente estructura:

- AirlineID: Identificador numérico de la aerolínea, de tipo entero.
- Name: Nombre de la aerolínea, una cadena.
- Alias: Abreviatura o acrónimo empleado para la aerolínea, en formato cadena.
- IATA: Código de dos letras, otra cadena.
- ICAO: Código de tres letras, en formato cadena.
- Callsign: Abreviatura empleada para llamadas a la aerolínea, código en cadena.
- Country: País de procedencia de la aerolínea, en formato cadena.
- Active: Estado de actividad de la aerolínea, un booleano.

Es especialmente notable la cantidad de información nula en esta tabla, aunque para este caso no precisará ningún tratamiento especial.

Procedimiento

En este segundo Sprint resumiré brevemente los pasos que sean similares a los realizados en el anterior procedimiento, deteniéndome en mayor medida en el uso de Confluent para emplear el ecosistema Kafka y en las consultas solicitadas.

Inicio de HDFS y comprobaciones

Para empezar, iniciamos HDFS en la terminal. Para ello, cambiamos al directorio de instalación de Hadoop y ejecutamos el comando de inicio del sistema distribuido de datos “[start-dfs.sh](#)”, tal como vimos en el anterior sprint.

```
bigdata@Big-Data:~$ cd /home/bigdata/hadoop-3.3.6/
bigdata@Big-Data:~/hadoop-3.3.6$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Big-Data]
bigdata@Big-Data:~/hadoop-3.3.6$
```

Para comprobar que los datos de la sesión anterior siguen apareciendo, vamos a listar en la terminal los datos que contiene la carpeta de ‘/data/aeropuertos’, usando el comando ‘dfs -ls’ seguido del nombre de la carpeta a mostrar:

```
bigdata@Big-Data:~/hadoop-3.3.6$ bin/hdfs dfs -ls /data/aeropuertos
Found 2 items
-rw-r--r--  1 bigdata supergroup          0 2025-05-21 09:28 /data/aeropuertos/_SUCCESS
-rw-r--r--  1 bigdata supergroup 435853 2025-05-21 09:28 /data/aeropuertos/datos_aeropuertos.parquet
bigdata@Big-Data:~/hadoop-3.3.6$
```

También podemos verlos en el WebUI que encontramos en el localhost:

Hadoop Overview Datanodes Datanode Volume Failures Snapshot Startup Progress Utilities ▾

Browse Directory

Show 25 ▾ entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	☐
☐	-rw-r--r--	bigdata	supergroup	0 B	May 21 09:28	1	128 MB	_SUCCESS	☐
☐	-rw-r--r--	bigdata	supergroup	425.64 KB	May 21 09:28	1	128 MB	datos_aeropuertos.parquet	☐

Showing 1 to 2 of 2 entries

Previous

1

Next

Hadoop, 2023.

Visto que los datos de la tabla empleada en la sesión anterior están correctamente cargados en el sistema HDFS, vamos a continuar con el uso de la plataforma Confluent.

Despliegue del ecosistema Kafka con Confluent

A continuación vamos a emplear la plataforma open-source de Confluent para arrancar los servicios que el ecosistema Kafka requiere para funcionar: Schema Registry, Zookeeper y el propio Kafka.

Primero vamos a cambiarnos al directorio donde se alojan los ficheros de la plataforma, para después usar el comando de confluent 'local services connect start', que arrancará el sistema y desplegará los servicios anteriormente mencionados.

```
bigdata@Big-Data:~$ cd /home/bigdata/confluent-7.5.1/
bigdata@Big-Data:~/confluent-7.5.1$ confluent local services connect start
The local commands are intended for a single-node development environment only
As of Confluent Platform 8.0, Java 8 will no longer be supported.

Using CONFLUENT_CURRENT: /tmp/confluent.079877
Starting ZooKeeper
ZooKeeper is [UP]
Starting Kafka
Kafka is [UP]
Starting Schema Registry
Schema Registry is [UP]
Starting Connect
Connect is [UP]
bigdata@Big-Data:~/confluent-7.5.1$
```

Con Kafka iniciado, es el momento de introducir el conector sink desde el parquet y cargarlo en Kafka-Connect. Vamos a comprobar primero la configuración del conector:

```
bigdata@Big-Data:~/confluent-7.5.1$ cat /home/bigdata/Descargas/hdfs3-parquet.json
{
  "name": "hdfs3-parquet",
  "config": {
    "connector.class": "io.confluent.connect.hdfs3.Hdfs3SinkConnector",
    "tasks.max": "1",
    "topics": "airlines_topic",
    "hdfs.url": "hdfs://localhost:9000",
    "flush.size": "10",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://localhost:8081",
    "confluent.topic.bootstrap.servers": "localhost:9092",
    "confluent.topic.replication.factor": "1",
    "format.class": "io.confluent.connect.hdfs3.parquet.ParquetFormat"
  }
}
```

Vemos que el conector está configurado para un máximo de una tarea simultánea, contiene el nombre del topic que crearemos y apunta al puerto donde establecerá la conexión, contando con un factor de replicación 1, es decir que no va a copiar la información más allá de nuestro sistema local.

Vista la configuración, procedemos a cargar el conector con el comando de confluent 'local services connect connector load', añadiendo el nombre del conector 'hdfs3-parquet', y seguido del json con la configuración, empleando el comando '--config' seguido de la ruta '/home/bigdata/Descargas/hdfs3-parquet.json'.

```
bigdata@Big-Data:~/confluent-7.5.1$ bin/confluent local services connect connector load hdfs3-parquet --config /home/bigdata/Descargas/hdfs3-parquet.json
The local commands are intended for a single-node development environment only, NOT for production usage. See more: https://docs.confluent.io/current/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

{
  "name": "hdfs3-parquet",
  "config": {
    "connector.class": "io.confluent.connect.hdfs3.Hdfs3SinkConnector",
    "tasks.max": "1",
    "topics": "airlines_topic",
    "hdfs.url": "hdfs://localhost:9000",
    "flush.size": "10",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://localhost:8081",
    "confluent.topic.bootstrap.servers": "localhost:9092",
    "confluent.topic.replication.factor": "1",
    "format.class": "io.confluent.connect.hdfs3.parquet.ParquetFormat",
    "name": "hdfs3-parquet"
  },
  "tasks": [],
  "type": "sink"
}
```

Al cargar vemos que nos indica las características de configuración, que coinciden con las que hemos visto previamente, además de mostrar el tipo de conector del que se trata, siendo de clase sink en este caso.

Ahora que hemos cargado el conector, vamos a comprobar que está operativo. Para ello usamos el comando de confluent 'local services connect connector status' seguido del nombre del conector: 'hdfs3-parquet'.

```
bigdata@Big-Data:~/confluent-7.5.1$ bin/confluent local services connect connector status hdfs3-parquet
The local commands are intended for a single-node development environment only, NOT for production usage.
As of Confluent Platform 8.0, Java 8 will no longer be supported.

{
  "name": "hdfs3-parquet",
  "connector": {
    "state": "RUNNING",
    "worker_id": "127.0.1.1:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "127.0.1.1:8083"
    }
  ],
  "type": "sink"
}
```

Visto que el conector está adecuadamente cargado y operativo, continuamos arrancando un consumidor datos de Kafka en formato Avro para el topic airlines_topic, tal como indica la configuración del conector.

Para arrancar el consumidor se ha de emplear el comando 'bin/kafka-avro-console-consumer', seguidamente indicamos la dirección del servicio de Kafka, local en este caso, con '--bootstrap-server localhost:9092' y por último el nombre del topic empleando el comando '--topic airlines_topic'.

```
bigdata@Big-Data:~/confluent-7.5.1$ bin/kafka-avro-console-consumer --bootstrap-server localhost:9092 --topic airlines_topic
[2025-05-20 14:22:18,578] INFO KafkaAvroDeserializerConfig values:
  auto.register.schemas = true
  avro.reflection.allow.null = false
  avro.use.logical.type.converters = false
  basic.auth.credentials.source = URL
  basic.auth.user.info = [hidden]
  bearer.auth.cache.expiry.buffer.seconds = 300
  bearer.auth.client.id = null
  bearer.auth.client.secret = null
  bearer.auth.credentials.source = STATIC_TOKEN
  bearer.auth.custom.provider.class = null
  bearer.auth.identity.pool.id = null
  bearer.auth.issuer.endpoint.url = null
  bearer.auth.logical.cluster = null
  bearer.auth.scope = null
  bearer.auth.scope.claim.name = scope
  bearer.auth.sub.claim.name = sub
  bearer.auth.token = [hidden]
  context.name.strategy = class io.confluent.kafka.serializers.context.NullContextNameStrategy
  http.connect.timeout.ms = 60000
  http.read.timeout.ms = 60000
  id.compatibility.strict = true
  key.subject.name.strategy = class io.confluent.kafka.serializers.subject.TopicNameStrategy
```

```
  schema.registry.ssl.keymanager.algorithm = SunX509
  schema.registry.ssl.keystore.certificate.chain = null
  schema.registry.ssl.keystore.key = null
  schema.registry.ssl.keystore.location = null
  schema.registry.ssl.keystore.password = null
  schema.registry.ssl.keystore.type = JKS
  schema.registry.ssl.protocol = TLSv1.2
  schema.registry.ssl.provider = null
  schema.registry.ssl.secure.random.implementation = null
  schema.registry.ssl.trustmanager.algorithm = PKIX
  schema.registry.ssl.truststore.certificates = null
  schema.registry.ssl.truststore.location = null
  schema.registry.ssl.truststore.password = null
  schema.registry.ssl.truststore.type = JKS
  schema.registry.url = [http://localhost:8081]
  specific.avro.key.type = null
  specific.avro.reader = false
  specific.avro.value.type = null
  use.latest.version = false
  use.latest.with.metadata = null
  use.schema.id = -1
  value.subject.name.strategy = class io.confluent.kafka.serializers.subject.TopicNameStrategy
(io.confluent.kafka.serializers.KafkaAvroDeserializerConfig:370)
```

Una vez arrancado, el consumidor muestra una larga lista de detalles de la configuración de kafka y el schema registry (aquí se muestra solo el inicio y el final de dicha lista). Este consumidor servirá para suministrar los datos hacia HDFS a través de Kafka.

A continuación vamos a observar los datos para asegurarnos que tienen la estructura especificada en las consideraciones generales, ya que tendremos que indicársela a Kafka cuando vaya a procesar los datos.

```
bigdata@Big-Data:~/confluent-7.5.1$ head /home/bigdata/Descargas/aerolineas.json
{"airlineID":1,"name":"Unknown","alias":"\\N","iata":"-","icao":"N/A","callsign":"\\N","country":"\\N","active":true}
{"airlineID":1,"name":"Private flight","alias":"\\N","iata":"-","icao":"N/A","callsign":"-","country":"-","active":true}
{"airlineID":2,"name":"135 Airways","alias":"\\N","iata":"-","icao":"GNL","callsign":"GENERAL","country":"United States","active":false}
{"airlineID":3,"name":"1Time Airline","alias":"\\N","iata":"1T","icao":"RNX","callsign":"NEXTIME","country":"South Africa","active":true}
{"airlineID":4,"name":"2 Sqn No 1 Elementary Flying Training School","alias":"\\N","iata":"-","icao":"WYT","callsign":"-","country":"United Kingdom","active":false}
{"airlineID":5,"name":"213 Flight Unit","alias":"\\N","iata":"-","icao":"TFU","callsign":"-","country":"Russia","active":false}
{"airlineID":6,"name":"223 Flight Unit State Airline","alias":"\\N","iata":"-","icao":"CHD","callsign":"CHKALOVSK-AVIA","country":"Russia","active":false}
{"airlineID":7,"name":"224th Flight Unit","alias":"\\N","iata":"-","icao":"TTF","callsign":"CARGO UNIT","country":"Russia","active":false}
{"airlineID":8,"name":"247 Jet Ltd","alias":"\\N","iata":"-","icao":"TWF","callsign":"CLOUD RUNNER","country":"United Kingdom","active":false}
{"airlineID":9,"name":"3D Aviation","alias":"\\N","iata":"-","icao":"SEC","callsign":"SECUREX","country":"United States","active":false}
```

Gracias a esto podemos comprobar que el fichero json contiene las columnas esperadas en el formato que hemos previsto, por lo que el siguiente paso será volcar la tabla de datos empleando un productor de Kafka en Avro, al que deberemos indicar el punto de conexión, el topic al que hacemos referencia y las propiedades de la tabla.

La llamada al productor se realiza con el comando 'bin/kafka-avro-console-producer', indicando el punto de conexión con '--broker-list localhost:9092' y el topic '--topic airlines_topic', añadiendo por último las propiedades de la tabla, '--property value.schema = '{"type": "record", "name": "airlines", "fields": [{"name": "airlineID", "type": "int"}, {"name": "name", "type": "string"}, {"name": "alias", "type": "string"}, {"name": "iata", "type": "string"}, {"name": "icao", "type": "string"}, {"name": "callsign", "type": "string"}, {"name": "country", "type": "string"}, {"name": "active", "type": "boolean"}]}'

Para completar el volcado de datos necesitamos usar una redirección de la línea de comandos, iniciando el comando cargando los datos desde la tabla en '/home/bigdata/Descargas/aerolineas.json' y redirigiéndola al productor anterior con '|'.

```
bigdata@Big-Data:~/confluent-7.5.1$ cat /home/bigdata/Descargas/aerolineas.json | bin/kafka-avro-console-producer --broker-list localhost:9092 --topic airlines_topic --property value.schema='{"type": "record", "name": "airlines", "fields": [{"name": "airlineID", "type": "int"}, {"name": "name", "type": "string"}, {"name": "alias", "type": "string"}, {"name": "iata", "type": "string"}, {"name": "icao", "type": "string"}, {"name": "callsign", "type": "string"}, {"name": "country", "type": "string"}, {"name": "active", "type": "boolean"}]}'
[2025-05-20 14:41:18,004] INFO KafkaAvroSerializerConfig values:
  auto.register.schemas = true
  avro.reflection.allow.null = false
  avro.remove.java.properties = false
  avro.use.logical.type.converters = false
  basic.auth.credentials.source = URL
  basic.auth.user.info = [hidden]
  bearer.auth.cache.expiry.buffer.seconds = 300
```

Si miramos la terminal de Confluent donde hemos lanzado el consumidor de avro y que estaba a la espera, vemos que comienzan a aparecer los datos de la tabla:

```
{
  "airlineID": 20961, "name": "Deutsche Luftverkehrsgesellschaft (DLI)", "alias": "-", "iata": "DW", "icao": "DWD", "callsign": "-", "country": "Germany", "active": false
},
{
  "airlineID": 20962, "name": "Orenburzhiye", "alias": "-", "iata": "-", "icao": "ORG", "callsign": "-", "country": "Russia", "active": true
},
{
  "airlineID": 20963, "name": "Atlantic Air Cargo", "alias": "-", "iata": "K3", "icao": "AWN", "callsign": "-", "country": "United States", "active": false
},
{
  "airlineID": 20976, "name": "World Scale Airlines", "alias": "World Scale", "iata": "W3", "icao": "MSS", "callsign": "-", "country": "United States", "active": true
},
{
  "airlineID": 20978, "name": "All America US", "alias": "All America US", "iata": "AG", "icao": "SSA", "callsign": "-", "country": "United States", "active": true
},
{
  "airlineID": 20986, "name": "University of Birmingham Air Squadron (RAF)", "alias": "BAEF RAF", "iata": "-", "icao": "UAV", "callsign": "UAV", "country": "United Kingdom", "active": true
},
{
  "airlineID": 20991, "name": "Choice Airways", "alias": "-", "iata": "-", "icao": "CSX", "callsign": "-", "country": "United States", "active": true
},
{
  "airlineID": 20994, "name": "Aruba Airlines", "alias": "-", "iata": "-", "icao": "ARU", "callsign": "-", "country": "Aruba", "active": true
},
{
  "airlineID": 20995, "name": "BudgetAir", "alias": "BudgetAir", "iata": "IK", "icao": "BG1", "callsign": "-", "country": "Germany", "active": true
},
{
  "airlineID": 20996, "name": "Dynamic Airways", "alias": "-", "iata": "-", "icao": "DVA", "callsign": "-", "country": "United States", "active": true
},
{
  "airlineID": 21012, "name": "Fly One", "alias": "Fly One", "iata": "F5", "icao": "FIS", "callsign": "-", "country": "Moldova", "active": true
},
{
  "airlineID": 21021, "name": "Nordica", "alias": "-", "iata": "EE", "icao": "N", "callsign": "REVAL", "country": "Estonia", "active": true
},
{
  "airlineID": 21036, "name": "Tez Jet Airlines", "alias": "-", "iata": "-", "icao": "TEZ", "callsign": "-", "country": "Kyrgyzstan", "active": true
},
{
  "airlineID": 21043, "name": "Kush Air", "alias": "-", "iata": "-", "icao": "KUH", "callsign": "-", "country": "South Sudan", "active": true
},
{
  "airlineID": 21056, "name": "Dummy", "alias": "-", "iata": "0E", "icao": "N", "callsign": "-", "country": "France", "active": false
},
{
  "airlineID": 21124, "name": "ViaAir", "alias": "Charter Air Transport", "iata": "-", "icao": "SRV", "callsign": "Stingray", "country": "United States", "active": true
},
{
  "airlineID": 21131, "name": "All America BOPY", "alias": "All America BOPY", "iata": "0P", "icao": "PYB", "callsign": "BOPY", "country": "Paraguay", "active": true
},
{
  "airlineID": 21179, "name": "Thai Vietjet Air", "alias": "-", "iata": "-", "icao": "TVJ", "callsign": "THAIVJET JET", "country": "Thailand", "active": true
},
{
  "airlineID": 21181, "name": "Air Andaman (2Y)", "alias": "-", "iata": "2Y", "icao": "AOM", "callsign": "AIR ANDAMAN", "country": "Thailand", "active": false
},
{
  "airlineID": 21240, "name": "TDA Toa Domestic Airlines", "alias": "TDA", "iata": "-", "icao": "TDA", "callsign": "Toa Domes", "country": "Japan", "active": false
},
{
  "airlineID": 21248, "name": "GX Airlines", "alias": "-", "iata": "-", "icao": "CBG", "callsign": "SPRAY", "country": "China", "active": true
},
{
  "airlineID": 21251, "name": "Lynx Aviation (L3SSX)", "alias": "-", "iata": "-", "icao": "SSX", "callsign": "Shasta", "country": "United States", "active": false
},
{
  "airlineID": 21266, "name": "Jetgo Australia", "alias": "-", "iata": "JG", "icao": "LW", "callsign": "-", "country": "Australia", "active": true
},
{
  "airlineID": 21276, "name": "Air Carnival", "alias": "-", "iata": "25", "icao": "N", "callsign": "-", "country": "India", "active": true
},
{
  "airlineID": 21317, "name": "Svyaz Rossiya", "alias": "Russian Commuter", "iata": "7R", "icao": "SJM", "callsign": "RussianConnecty", "country": "Russia", "active": true
}
```

Con esto los datos deberían haberse volcado, particionados según la ventana que se estableciera a Kafka, al sistema HDFS. Vamos a comprobarlo viendo el fichero del topic indicado, usando el comando 'bin/hdfs dfs -ls' seguido de la ubicación de los datos '/topics/airlines_topic/partition=0':

```
bigdata@Big-Data:~/hadoop-3.3.6$ bin/hdfs dfs -ls /topics/airlines_topic/partition=0
-rw-r--r-- 3 bigdata supergroup 2392 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005130+0000005139.parquet
-rw-r--r-- 3 bigdata supergroup 2457 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005140+0000005149.parquet
-rw-r--r-- 3 bigdata supergroup 2353 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005150+0000005159.parquet
-rw-r--r-- 3 bigdata supergroup 2504 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005160+0000005169.parquet
-rw-r--r-- 3 bigdata supergroup 2422 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005170+0000005179.parquet
-rw-r--r-- 3 bigdata supergroup 2449 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005180+0000005189.parquet
-rw-r--r-- 3 bigdata supergroup 2530 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005190+0000005199.parquet
-rw-r--r-- 3 bigdata supergroup 2404 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005200+0000005209.parquet
-rw-r--r-- 3 bigdata supergroup 2378 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005210+0000005219.parquet
-rw-r--r-- 3 bigdata supergroup 2418 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005220+0000005229.parquet
-rw-r--r-- 3 bigdata supergroup 2474 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005230+0000005239.parquet
-rw-r--r-- 3 bigdata supergroup 2405 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005240+0000005249.parquet
-rw-r--r-- 3 bigdata supergroup 2304 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005250+0000005259.parquet
-rw-r--r-- 3 bigdata supergroup 2458 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005260+0000005269.parquet
-rw-r--r-- 3 bigdata supergroup 2435 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005270+0000005279.parquet
-rw-r--r-- 3 bigdata supergroup 2411 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005280+0000005289.parquet
-rw-r--r-- 3 bigdata supergroup 2385 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005290+0000005299.parquet
-rw-r--r-- 3 bigdata supergroup 2426 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005300+0000005309.parquet
-rw-r--r-- 3 bigdata supergroup 2454 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005310+0000005319.parquet
-rw-r--r-- 3 bigdata supergroup 2390 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005320+0000005329.parquet
-rw-r--r-- 3 bigdata supergroup 2365 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005330+0000005339.parquet
-rw-r--r-- 3 bigdata supergroup 2350 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005340+0000005349.parquet
-rw-r--r-- 3 bigdata supergroup 2443 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005350+0000005359.parquet
-rw-r--r-- 3 bigdata supergroup 2344 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005360+0000005369.parquet
-rw-r--r-- 3 bigdata supergroup 2306 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005370+0000005379.parquet
-rw-r--r-- 3 bigdata supergroup 2383 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005380+0000005389.parquet
-rw-r--r-- 3 bigdata supergroup 2347 2025-05-20 14:49 /topics/airlines_topic/partition=0/airlines_topic+0+0000005390+0000005399.parquet
```

Podemos comprobar que aparece una larga lista de particiones, tantas como Kafka ha generado, en base al número de entradas en la tabla y a la ventana que hubiéramos configurado.

Hadoop

Overview

Datanodes

Datanode Volume Failures

Snapshot

Startup Progress

Utilities

Browse Directory

/topics

Go!

Show

25

entries

Search:

	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
☐	drwxr-xr-x	bigdata	supergroup	0 B	May 20 14:47	0	0 B	+tmp
☐	drwxr-xr-x	bigdata	supergroup	0 B	May 20 14:47	0	0 B	airlines_topic
☐	drwxr-xr-x	bigdata	supergroup	0 B	May 20 13:41	0	0 B	fruits_topic

Showing 1 to 3 of 3 entries

Previous

1

Next

Hadoop, 2023.

También podemos acudir al explorador de HDFS en el webUI, que nos permite ver que se ha creado tanto el topic ‘airlines_topic’ como los archivos en su interior correspondientes a todas las particiones de los datos consumidos a través del conector de Kafka.

Browse Directory

/topics/airlines_topic/partition=0

Go!

Show

25

entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	-rw-r--r--	bigdata	supergroup	2.24 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000000+0000000009.parquet	
☐	-rw-r--r--	bigdata	supergroup	2.33 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000010+0000000019.parquet	
☐	-rw-r--r--	bigdata	supergroup	2.41 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000020+0000000029.parquet	
☐	-rw-r--r--	bigdata	supergroup	2.39 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000030+0000000039.parquet	
☐	-rw-r--r--	bigdata	supergroup	2.48 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000040+0000000049.parquet	
☐	-rw-r--r--	bigdata	supergroup	2.28 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000050+0000000059.parquet	
☐	-rw-r--r--	bigdata	supergroup	2.46 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000060+0000000069.parquet	
☐	-rw-r--r--	bigdata	supergroup	2.31 KB	May 20 14:47	3	256 MB	airlines_topic+0+0000000070+0000000079.parquet	

Creación de la tabla externa y consultas

Con los datos de nuestra segunda tabla ya cargados en HDFS podemos, en primer lugar, detener el consumidor con la combinación de teclas Control + C estando ubicados en su consola correspondiente, en segundo lugar, proceder a crear la tabla en Hive.

Para ello, nos colocaremos en el directorio de instalación de Hive y lo iniciaremos con el ya conocido comando 'bin/beeline -u jdbc:hive2://'

```
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 3.1.3 by Apache Hive
0: jdbc:hive2://>
```

Una vez arrancado Hive y estando disponibles los comandos de HQL, procedemos a crear la tabla externa que apunte a los datos recién introducidos, siguiendo el mismo esquema tabular indicado al inicio, e indicando la ruta donde se almacenan los datos.

```

Connected to: Apache Hive (version 3.1.3)
Driver: Hive JDBC (version 3.1.3)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 3.1.3 by Apache Hive
0: jdbc:hive2://> CREATE EXTERNAL TABLE aerolineas(
. . . . . > airlineID int,
. . . . . > name string,
. . . . . > alias string,
. . . . . > iata string,
. . . . . > icao string,
. . . . . > callsign string,
. . . . . > country string,
. . . . . > active boolean)
. . . . . > STORED AS parquet
. . . . . > LOCATION 'topics/airlines_topic/partition=0';
OK
No rows affected (1,191 seconds)
0: jdbc:hive2://> SHOW TABLES
. . . . . > ;
OK
+-----+
| tab_name |
+-----+
| aerolineas |
| aeropuertos |
| fruits |
+-----+
3 rows selected (0,387 seconds)
0: jdbc:hive2://>

```

Con el proceso mostrado previamente creamos la nueva tabla y mostramos todas las tablas actualmente disponibles para Hive en nuestro sistema HDFS. Vemos que tenemos la tabla 'aeropuertos' del sprint 1, así como la nueva tabla 'aerolineas' que acabamos de crear.

```

0: jdbc:hive2://> SELECT * FROM aerolineas
. . . . . > LIMIT 10;
OK
+-----+-----+-----+-----+-----+-----+-----+-----+
| aerolineas.airline_id | aerolineas.name | aerolineas.alias | aerolineas.iata | aerolineas.icao | aerolineas.callsign | aerolineas.country | aerolineas.active |
+-----+-----+-----+-----+-----+-----+-----+-----+
| NULL | Unknown | \N | - | N/A | \N | \N | true |
| NULL | Private flight | \N | - | N/A | - | - | true |
| NULL | 135 Airways | \N | - | GNL | GENERAL | United States | false |
| NULL | 17me Airline | \N | 1T | RNK | NEXTIME | South Africa | true |
| NULL | 2 Sqn No 1 Elementary Flying Training School | \N | - | WYT | - | United Kingdom | false |
| NULL | 213 Flight Unit | \N | - | TFU | - | Russia | false |
| NULL | 223 Flight Unit State Airline | \N | - | CHD | CHKALOVSK-AVIA | Russia | false |
| NULL | 224th Flight Unit | \N | - | TTF | CARGO UNIT | Russia | false |
| NULL | 247 Jet Ltd | \N | - | TWF | CLOUD RUNNER | United Kingdom | false |
| NULL | 3D Aviation | \N | - | SEC | SECUREX | United States | false |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Aquí mostramos una pequeña lista con las primeras 10 entradas de la tabla, para comprobar que se ha creado correctamente y es accesible por Hive. Vamos a continuación a responder a las tres preguntas planteadas mediante consultas a las tablas que tenemos disponibles.

Pregunta 1: ¿Cuántas aerolíneas tiene en total Estados Unidos?

Para esta primera pregunta simplemente vamos a extraer los datos de los países de procedencia de las aerolíneas, realizando un conteo sobre los mismos en el caso en que la entrada corresponda a Estados Unidos.

Consulta:

```
0: jdbc:hive2://> SELECT COUNT(country)
. . . . . > FROM aerolineas
. . . . . > WHERE country = 'United States';
```

Respuesta:

```
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 3408584 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
+-----+
| _c0 |
+-----+
| 1099 |
+-----+
1 row selected (30.736 seconds)
```

Tal como muestra la tabla, vemos que los EEUU tienen un total de 1099 aerolíneas.

Pregunta 2: ¿Cuáles son los 10 países con más aerolíneas inactivas?

Para esta segunda consulta necesitamos, además de los datos del país de origen de la aerolínea, su estado, sobre el que vamos a realizar un conteo en el caso de que este sea inactivo (active = False). Posteriormente realizaremos una agrupación por país en orden descendente, y mostraremos los 10 primeros.

Consulta:

```
0: jdbc:hive2://> SELECT COUNT(active) as INACTIVE, country
. . . . . > FROM aerolineas
. . . . . > WHERE active = False
. . . . . > GROUP BY country
. . . . . > ORDER BY INACTIVE DESC
. . . . . > LIMIT 10;
```

Respuesta:

```
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 3435298 HDFS Write: 0 SUCCESS
Stage-Stage-2:  HDFS Read: 3435298 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
+-----+-----+
| inactive | country |
+-----+-----+
| 943      | United States |
| 427      | Mexico       |
| 369      | United Kingdom |
| 286      | Canada       |
| 158      | Russia       |
| 142      | Spain        |
| 98       | France       |
| 97       | Germany      |
| 81       | South Africa |
| 80       | Nigeria      |
+-----+-----+
10 rows selected (23,114 seconds)
```

Aquí tenemos la lista de los 10 países con mayor número de aerolíneas inactivas, que son, por orden descendente: EEUU, Méjico, Reino Unido, Canadá, Rusia, España, Francia, Alemania, Sudáfrica y Nigeria.

Pregunta 3: ¿Qué países tienen aerolíneas en activo y aeropuertos con una latitud mayor de 80?

Esta tercera y última consulta es la más compleja, pues implica recabar y cruzar datos tanto de la tabla realizada en este sprint como la del sprint previo. Para ello hemos de tener claro que, cuando empleamos varias tablas a la vez, en HQL nos referimos a cada columna como 'nombre de la tabla'. 'nombre de la columna'. En este caso, vamos a recabar los datos de países de origen de las aerolíneas, cruzarlo con los de la tabla aeropuertos y establecer ambos condicionales respecto a las dos tablas:

Consulta:

```
0: jdbc:hive2://> SELECT aerolineas.country
. . . . . > FROM aerolineas
. . . . . > JOIN aeropuertos ON aerolineas.country = aeropuertos.country
. . . . . > WHERE aerolineas.active = True AND aeropuertos.latitude > 80
. . . . . > GROUP BY aerolineas.country;
```

Respuesta:

```
2025-05-21 09:38:01,711 Stage-2 map = 100%, reduce = 100%
Ended Job = job_local340275910_0002
MapReduce Jobs Launched:
Stage-Stage-2: HDFS Read: 8629152 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
+-----+
| aerolineas.country |
+-----+
| Canada             |
| New Zealand        |
| Russia              |
+-----+
3 rows selected (29,865 seconds)
```

De acuerdo con esta consulta, los tres países que reúnen ambas condiciones son Canadá, Nueva Zelanda y Rusia. Ante la extrañeza que me generó el caso de Nueva Zelanda, decidí mostrar los aeropuertos en cuestión:

```
0: jdbc:hive2://> SELECT DISTINCT aerolineas.country, aeropuertos.name, aeropuertos.latitude, aeropuertos.longitude
. . . . . > FROM aerolineas
. . . . . > JOIN aeropuertos ON aerolineas.country = aeropuertos.country
. . . . . > WHERE aerolineas.active = True AND aeropuertos.latitude > 80;

2025-05-21 09:53:20,156 Stage-2 map = 100%, reduce = 100%
Ended Job = job_local1413709398_0004
MapReduce Jobs Launched:
Stage-Stage-2: HDFS Read: 15499748 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
```

aerolineas.country	aeropuertos.name	aeropuertos.latitude	aeropuertos.longitude
Canada	Alert Airport	82.51779937740001	-62.2806015015
New Zealand	[Duplicate] Illertissen see EDMI - ED-0425 location moved out-of-the way	89.5	1.0E-4
Russia	Nagurskoye	80.803207	47.663586

Al parecer esto se debe a que hay dos formas de registrar la latitud, y dado que la longitud sí empleaba números negativos para la mitad al oeste del meridiano de Greenwich del globo, esperaba que hicieran lo mismo para las latitudes al sur del ecuador, pero al parecer no es el caso y emplean un entero sin signo. Esto puede llevar a errores de coordenadas, dado que no hacen distinción entre Norte y Sur en cuanto a la medida de latitud.

Conclusiones

Esta práctica nos permite comprobar el funcionamiento de los conectores para un sistema de ingesta y procesamiento de datos en streaming como es Kafka, así como su conjugación con un sistema de archivos distribuido como HDFS, pensado para la capa de permanencia o batch.

Además introduce el mecanismo de despliegue de estos servicios dentro de su ecosistema, con herramientas open-source como es el caso de Confluent para el manejo de conectores

y el arranque de los consumidores y productores de avro. También nos permite practicar con consultas más complejas en Hive, haciendo uso de HQL.